

SENAI CENTROWEG

Desenvolvimento de Interfaces para Visualização de Dados

TRABALHO DE PESQUISA E APLICAÇÃO
PRÁTICA

SISTEMA DE CONTROLE DE QUALIDADE
PARA LINHA DE MONTAGEM DE MOTORES
ELÉTRICOS

Integrantes:

Adriel Soares

André Matteussi

Cristian Sabini

Vinicius Goedert

Jaraguá do Sul– SC
2026

Sumário

1	Introdução	3
2	Contextualização da Aplicação	3
3	Problema de Aplicação	4
4	Objetivos	5
4.1	Objetivo Geral	5
4.2	Objetivos Específicos	5
5	Modelo dos Dados	5
6	Exemplo do Dataset	6
7	Arquitetura de Software	6
7.1	Arquivos de Configuração e Documentação	8
7.2	Camada de Dados	9
7.3	Processo ETL	9
7.4	Backend	9
7.4.1	Arquivo main.py	10
7.4.2	Arquivo config.py	10
7.4.3	Arquivo database.py	10
7.5	Schemas	10
7.6	Services	10
7.7	Routers	11
7.8	Frontend	11
7.8.1	Arquivo index.html	12
7.8.2	Arquivos CSS	12
7.8.3	Arquivos JavaScript	12
7.9	Fluxo entre as Camadas	13

8 Pipeline da Aplicação	14
9 Tratamento dos Dados	14
10 Armazenamento em Parquet	15
11 Consulta dos Dados com DuckDB	15
12 API REST com FastAPI	16
13 Interface Web	16
14 Visualização dos Dados com Chart.js	16
14.1 Justificativa para Utilização do Chart.js	17
14.2 Integração entre API e Chart.js	18
14.3 Aplicação dos Gráficos no Dashboard	19
14.4 Responsabilidade de Cada Tecnologia	20
15 Dashboard de Controle de Qualidade	20
15.1 Diagrama de Pareto	21
15.2 Evolução do Scrap por Lote	21
15.3 Causas por Turno	21
16 Fluxo de Utilização	21
17 Conclusão	22
Referências	23

1 Introdução

A visualização de dados possui papel relevante na análise de processos industriais, especialmente em ambientes nos quais são geradas grandes quantidades de informações relacionadas à produção e ao controle de qualidade. A transformação de dados brutos em informações visuais permite identificar padrões, anomalias e tendências que podem auxiliar profissionais na interpretação do processo produtivo.

No contexto proposto para este trabalho, será desenvolvido um sistema web destinado à análise de dados de controle de qualidade de uma linha de montagem de motores elétricos. A aplicação será estruturada a partir de um conjunto de dados simulado, contendo informações relacionadas a lotes de produção, modelos de motores, componentes, turnos, defeitos, causas e quantidade de peças destinadas ao retrabalho ou ao refugo.

O projeto corresponde ao tema destinado ao Grupo 3, definido como *Gestão do Controle de Qualidade, Peças Defeituosas e Refugo*. De acordo com a proposta do trabalho, a aplicação deverá possibilitar a construção de um diagrama de Pareto interativo, apresentar a evolução temporal do refugo por lote e permitir a classificação das causas-raiz de acordo com o turno de produção.

Dessa forma, o trabalho busca integrar conceitos de tratamento de dados, armazenamento colunar, consultas SQL, desenvolvimento de APIs e visualização de dados em uma única aplicação web.

2 Contextualização da Aplicação

A aplicação será desenvolvida considerando um cenário industrial simulado inspirado no setor de montagem de motores elétricos das famílias W12 e W22 da WEG.

O objetivo não é reproduzir o processo produtivo real da empresa nem utilizar dados internos da organização. O contexto da WEG será utilizado como referência para representar um ambiente industrial plausível no qual uma aplicação de análise de qualidade poderia ser empregada.

No cenário proposto, uma linha de montagem produz diferentes modelos de motores elétricos organizados em lotes e ordens de produção. Durante o processo são realizadas inspeções de qualidade para identificar possíveis não conformidades nos componentes e nos motores montados.

Entre os componentes considerados no dataset estão elementos como carcaças, rolamentos e caixas de ligação. Para cada ocorrência de qualidade poderão ser registradas informações referentes ao lote, modelo do motor, componente, estação de montagem, turno,

tipo de defeito, causa, causa-raiz e destino da peça.

As ocorrências identificadas durante a inspeção poderão resultar em três situações principais:

- aprovação da peça;
- encaminhamento para retrabalho;
- encaminhamento para refugo (scrap).

Os dados utilizados na aplicação serão simulados e terão finalidade exclusivamente acadêmica. Portanto, os valores apresentados no dataset não representam registros reais da WEG.

3 Problema de Aplicação

Em um ambiente industrial, a existência de registros de qualidade não garante, por si só, que as informações sejam facilmente interpretadas pelos responsáveis pelo processo.

Quando os dados estão distribuídos em arquivos ou registros estruturados, pode ser necessário realizar diferentes consultas para descobrir quais defeitos ocorrem com maior frequência, quais lotes apresentam maior quantidade de refugo ou quais causas estão mais associadas a determinado turno.

Diante desse cenário, o problema abordado pelo projeto consiste em desenvolver uma aplicação capaz de transformar registros de qualidade em informações visuais e interativas.

A aplicação deverá permitir que o usuário identifique rapidamente:

- quais são os principais tipos de defeito;
- como o volume de refugo varia entre os lotes;
- quais são as principais causas-raiz;
- como as causas dos defeitos se distribuem entre os diferentes turnos;
- quais componentes e etapas de montagem apresentam maior quantidade de ocorrências.

4 Objetivos

4.1 Objetivo Geral

Desenvolver uma aplicação web para visualização e análise de dados simulados de controle de qualidade de uma linha de montagem de motores elétricos, utilizando processamento de dados, armazenamento em formato Parquet, consultas SQL, API REST e visualizações interativas.

4.2 Objetivos Específicos

- Criar um conjunto de dados simulados relacionado ao processo de controle de qualidade;
- Realizar a limpeza e o tratamento dos dados utilizando Python e Pandas;
- Armazenar os dados tratados no formato Parquet;
- Utilizar DuckDB para realizar consultas SQL diretamente sobre os dados armazenados;
- Desenvolver uma API REST utilizando FastAPI;
- Disponibilizar os resultados das consultas em formato JSON;
- Desenvolver uma interface utilizando HTML5, CSS3 e JavaScript;
- Utilizar Chart.js para a construção dos gráficos interativos;
- Desenvolver um diagrama de Pareto para os defeitos;
- Apresentar a evolução do refugo por lote;
- Apresentar a distribuição das causas por turno.

5 Modelo dos Dados

Os dados iniciais da aplicação serão armazenados em um arquivo CSV (*Comma-Separated Values*). O dataset foi estruturado para conter informações suficientes para realizar as análises propostas para o Grupo 3.

A Tabela 1 apresenta as colunas previstas para o arquivo.

Tabela 1: Modelo de dados utilizado no arquivo CSV

Coluna	Exemplo	Finalidade
data	2026-09-15	Análise temporal
lote	L20260915-01	Identificação do lote
ordem_producao	OP45821	Identificação da produção
modelo_motor	W22	Separação dos modelos
codigo_peca	CAR-W22-160	Identificação do componente
peca	Carcaca	Análise por componente
turno	1º Turno	Análise por turno
estacao_montagem	Montagem Carcaca	Localização da etapa
quantidade_produzida	500	Volume produzido
quantidade_inspecionada	100	Amostragem/inspeção
quantidade_aprovada	94	Peças aprovadas
quantidade_defeituosa	6	Peças com defeito
quantidade_retrabalho	4	Peças recuperáveis
quantidade_refugo	2	Peças destinadas ao scrap
tipo_defeito	Dimensão	Classificação do defeito
causa	Montagem	Causa do problema
causa_raiz	Torque inadequado	Análise da causa-raiz
severidade	Média	Classificação do problema
status	Retrabalho	Destino da peça

Fonte: Elaborado pelos autores (2026).

6 Exemplo do Dataset

Um registro do dataset poderá apresentar a seguinte estrutura:

```
data,lote,ordem_producao,modelo_motor,codigo_peca,peca,turno,
estacao_montagem,quantidade_produzida,quantidade_inspecionada,
quantidade_aprovada,quantidade_defeituosa,quantidade_retrabalho,
quantidade_refugo,tipo_defeito,causa,causa_raiz,severidade,status

2026-09-15,L20260915-01,OP45821,W22,CAR-W22-160,Carcaca,
1o Turno,Montagem Carcaca,500,100,94,6,4,2,Dimensao,
Montagem,Torque inadequado,Media,Retrabalho
```

O dataset será posteriormente submetido a uma etapa de processamento para validação, limpeza, transformação e preparação para as consultas analíticas.

7 Arquitetura de Software

A aplicação foi organizada em uma estrutura de diretórios que separa as responsabilidades relacionadas ao processamento dos dados, à camada de serviços da API e à

interface de visualização. Essa organização tem como objetivo facilitar a manutenção do protótipo, permitindo identificar de forma clara onde estão localizados os dados, os scripts de processamento, os componentes do backend e os arquivos responsáveis pela interface.

A arquitetura foi definida considerando o caráter pedagógico do projeto, mantendo uma estrutura modular sem introduzir complexidade desnecessária. O projeto utiliza Python no processamento dos dados e no backend, enquanto a interface é desenvolvida utilizando HTML, CSS e JavaScript, com a biblioteca Chart.js para a construção das visualizações.

A estrutura geral do projeto é apresentada a seguir.

Listing 1: Estrutura de diretórios do projeto

```
controle-qualidade/
|
+-- .gitignore
+-- README.md
+-- requirements.txt
|
+-- data/
|   +-- raw/
|   |   +-- .gitkeep
|   |   +-- producao_motores.csv
|   |
|   +-- processed/
|       +-- .gitkeep
|       +-- qualidade.parquet
|
+-- etl/
|   +-- __init__.py
|   +-- process_data.py
|
+-- backend/
|   +-- app/
|       +-- __init__.py
|       +-- main.py
|       +-- config.py
|       +-- database.py
|       |
|       +-- schemas/
|           +-- __init__.py
|           +-- quality_schemas.py
|       |
|       +-- services/
|           +-- __init__.py
|           +-- analytics_service.py
|       |
|
```

```
|      +-- routers/
|      +-- __init__.py
|      +-- pareto.py
|      +-- scrap.py
|      +-- causes.py
|
+-- frontend/
    +-- index.html
    |
    +-- css/
    |   +-- variables.css
    |   +-- styles.css
    |
    +-- js/
    |   +-- config.js
    |   |
    |   +-- services/
    |   |   +-- api.js
    |   |   |
    |   +-- charts/
    |       +-- paretoChart.js
    |       +-- scrapChart.js
    |       +-- causesChart.js
    |       +-- main.js
```

7.1 Arquivos de Configuração e Documentação

Na raiz do projeto estão localizados os arquivos utilizados para configuração, documentação e gerenciamento das dependências da aplicação.

O arquivo `.gitignore` contém as regras utilizadas pelo Git para impedir que arquivos que não devem fazer parte do versionamento sejam adicionados ao repositório, como ambientes virtuais, arquivos temporários, caches e dados que não sejam apropriados para versionamento.

O arquivo `README.md` contém a documentação básica do projeto, incluindo informações sobre a arquitetura, instalação das dependências e instruções necessárias para executar a aplicação.

O arquivo `requirements.txt` registra as principais dependências Python utilizadas no projeto, incluindo `fastapi`, `uvicorn`, `duckdb`, `pandas` e `pyarrow`.

7.2 Camada de Dados

A pasta `data/` é responsável pelo armazenamento dos dados utilizados pela aplicação. Ela é dividida em duas partes:

- `data/raw/`: contém os dados originais utilizados como entrada do processo de tratamento;
- `data/processed/`: contém os dados após o processamento e tratamento realizado pelo ETL.

O arquivo `producao_motores.csv`, localizado em `data/raw/`, representa o dataset original simulado, contendo as informações utilizadas no cenário de controle de qualidade dos motores.

Após o processamento, os dados são armazenados no arquivo `qualidade.parquet`, localizado em `data/processed/`. Esse arquivo representa a versão tratada e armazenada em formato colunar utilizada pelas consultas analíticas.

A separação entre os diretórios `raw` e `processed` permite preservar o arquivo original e diferenciar os dados de entrada dos dados preparados para análise.

7.3 Processo ETL

A pasta `etl/` contém os arquivos responsáveis pelo processo de Extração, Transformação e Carga (ETL) dos dados.

O arquivo principal dessa etapa é o `process_data.py`, desenvolvido utilizando Python e Pandas. Sua responsabilidade é carregar o arquivo CSV, realizar a higienização e tipagem dos dados e gerar o arquivo Parquet que será utilizado posteriormente pelo DuckDB.

O fluxo simplificado dessa etapa é:

`producao_motores.csv` → **Pandas** → `qualidade.parquet`

Dessa forma, o ETL estabelece a ligação entre os dados brutos e a camada analítica da aplicação.

7.4 Backend

A pasta `backend/` concentra a implementação da API responsável por disponibilizar os dados para o frontend.

A aplicação é organizada dentro da pasta `backend/app/`, que contém os componentes principais da API desenvolvida com FastAPI.

7.4.1 Arquivo `main.py`

O arquivo `main.py` representa o ponto de entrada da API. Nele é criada a instância do FastAPI, são configurados os middlewares necessários, como CORS, e são incluídos os diferentes routers da aplicação.

7.4.2 Arquivo `config.py`

O arquivo `config.py` concentra configurações utilizadas pela aplicação, incluindo variáveis de ambiente e caminhos relacionados aos arquivos de dados.

Essa separação permite evitar que caminhos e configurações sejam espalhados pelos diferentes componentes do backend.

7.4.3 Arquivo `database.py`

O arquivo `database.py` é responsável pelo acesso ao DuckDB.

O DuckDB será utilizado como mecanismo de consulta analítica dos dados armazenados no arquivo Parquet. Dessa forma, as consultas SQL realizadas pela aplicação poderão acessar os dados tratados sem a necessidade de utilizar um servidor de banco de dados tradicional.

7.5 Schemas

A pasta `schemas/` contém os modelos utilizados para validação e tipagem dos dados retornados pela API.

O arquivo `quality_schemas.py` utilizará modelos Pydantic para definir a estrutura esperada das respostas JSON.

Essa camada contribui para manter um formato consistente para os dados disponibilizados pelos endpoints da aplicação.

7.6 Services

A pasta `services/` concentra a lógica analítica da aplicação.

O arquivo `analytics_service.py` será responsável por executar e organizar as consultas SQL utilizadas para produzir os dados necessários às visualizações.

Entre as consultas previstas estão aquelas relacionadas ao diagrama de Pareto, à evolução do scrap por lote e à classificação das causas por turno.

A separação da lógica analítica em um serviço próprio evita que as consultas SQL fiquem diretamente misturadas com a definição dos endpoints.

7.7 Routers

A pasta `routers/` organiza os endpoints da API de acordo com a finalidade de cada conjunto de dados.

Foram definidos três módulos principais:

- `pareto.py`: contém os endpoints relacionados ao Pareto e aos defeitos;
- `scrap.py`: contém os endpoints relacionados ao refugo e à evolução do scrap por lote;
- `causes.py`: contém os endpoints relacionados às causas e à análise das causas por turno.

A divisão resulta nos seis endpoints principais previstos para a aplicação:

Tabela 2: Endpoints principais da API

Endpoint	Finalidade
<code>/api/pareto</code>	Dados para construção do diagrama de Pareto.
<code>/api/defeitos</code>	Consulta dos tipos e quantidades de defeitos.
<code>/api/scrap</code>	Informações gerais relacionadas ao refugo.
<code>/api/scrap/lotes</code>	Evolução do refugo de acordo com os lotes.
<code>/api/causas</code>	Consulta das causas identificadas.
<code>/api/causas/turnos</code>	Classificação das causas de acordo com o turno.

Fonte: Elaborado pelos autores (2026).

7.8 Frontend

A pasta `frontend/` contém os arquivos responsáveis pela interface gráfica da aplicação.

A interface é dividida entre HTML, CSS e JavaScript, mantendo separadas as responsabilidades de estrutura, apresentação e comportamento.

7.8.1 Arquivo `index.html`

O arquivo `index.html` representa a estrutura principal da interface do dashboard.

Nele serão definidos os elementos HTML utilizados para apresentar os indicadores de desempenho (KPIs), filtros, títulos e contêineres destinados aos gráficos.

7.8.2 Arquivos CSS

A pasta `css/` contém os arquivos responsáveis pela apresentação visual da aplicação.

O arquivo `variables.css` concentra os tokens de design, como cores, tipografia e espaçamentos utilizados pela interface.

O arquivo `styles.css` contém as regras de layout e estilização dos componentes, incluindo a utilização de CSS Grid e Flexbox para organização e responsividade da interface.

7.8.3 Arquivos JavaScript

A pasta `js/` concentra a lógica de funcionamento da interface.

O arquivo `config.js` contém as configurações utilizadas pelo frontend, incluindo a URL base da API.

A pasta `services/` contém o arquivo `api.js`, responsável pelas requisições assíncronas realizadas utilizando `fetch`.

A pasta `charts/` concentra os componentes responsáveis pelas visualizações utilizando `Chart.js`.

O arquivo `paretoChart.js` será responsável pela configuração do diagrama de Pareto, combinando as barras das quantidades de defeitos com a linha de percentual acumulado.

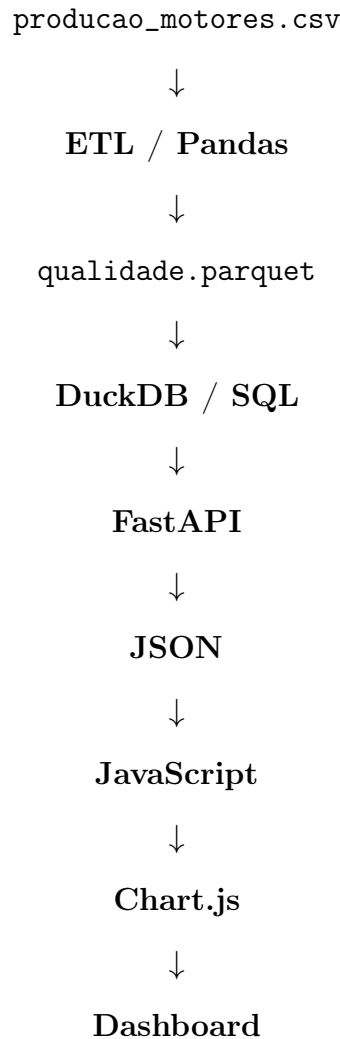
O arquivo `scrapChart.js` será responsável pelo gráfico de linha utilizado para representar a evolução temporal do refugo por lote.

O arquivo `causesChart.js` será responsável pelo gráfico de barras agrupadas utilizado para comparar as causas dos problemas entre os diferentes turnos.

Por fim, o arquivo `main.js` será responsável pela inicialização da interface, carregamento dos KPIs, configuração dos listeners dos filtros e coordenação das diferentes visualizações.

7.9 Fluxo entre as Camadas

A organização dos diretórios representa também o fluxo lógico da aplicação. Os dados percorrem as diferentes camadas da seguinte maneira:



Essa organização permite que cada componente tenha uma responsabilidade definida. O ETL é responsável pela preparação dos dados, o Parquet pelo armazenamento tratado, o DuckDB pelas consultas analíticas, o FastAPI pela disponibilização dos dados e o frontend pela interação e visualização das informações.

Para um protótipo de finalidade pedagógica, essa divisão oferece um nível de modularização suficiente para demonstrar a arquitetura completa da solução sem introduzir uma quantidade excessiva de camadas ou abstrações.

8 Pipeline da Aplicação

A Figura 1 apresenta o pipeline completo da aplicação, desde os dados simulados até a disponibilização das informações no dashboard.



Figura 1: Pipeline da aplicação de controle de qualidade.

Fonte: Elaborado pelos autores (2026).

9 Tratamento dos Dados

Após a disponibilização do arquivo CSV, os dados serão carregados utilizando a biblioteca Pandas.

A etapa de tratamento terá como finalidade garantir que os dados estejam em condições adequadas para armazenamento e análise.

Entre as operações previstas estão:

- identificação e tratamento de valores ausentes;
- verificação de registros duplicados;
- padronização das categorias;

- conversão dos tipos de dados;
- validação das quantidades;
- criação de métricas derivadas;
- preparação dos dados para as consultas SQL.

Como resultado dessa etapa, será produzido um conjunto de dados estruturado e consistente para armazenamento em formato Parquet.

10 Armazenamento em Parquet

Os dados tratados serão armazenados no formato Parquet. Diferente do CSV, que organiza os dados de maneira textual, o Parquet utiliza uma estrutura colunar, sendo adequado para operações analíticas sobre dados estruturados.

Nesse projeto, o Parquet será utilizado como formato de armazenamento dos dados tratados antes da etapa de consulta.

A utilização desse formato também permite demonstrar a diferença entre o arquivo original, utilizado como fonte dos dados, e o arquivo tratado e otimizado utilizado nas etapas posteriores do pipeline.

11 Consulta dos Dados com DuckDB

O DuckDB será utilizado como mecanismo de consulta analítica, permitindo executar comandos SQL diretamente sobre os arquivos Parquet.

Essa etapa será responsável por realizar operações como agrupamentos, filtros, ordenações, somas e contagens.

Um exemplo de consulta é apresentado a seguir:

```
SELECT
    tipo defeito,
    SUM(quantidade defeituosa) AS quantidade
FROM 'qualidade.parquet'
GROUP BY tipo defeito
ORDER BY quantidade DESC;
```

O resultado dessa consulta poderá ser utilizado pela aplicação para construir a visualização dos principais defeitos.

Outras consultas serão desenvolvidas para obter os dados necessários para a análise do scrap por lote e para a classificação das causas por turno.

12 API REST com FastAPI

Após a consulta dos dados, os resultados serão disponibilizados para a interface por meio de uma API REST desenvolvida utilizando FastAPI.

A API funcionará como uma camada intermediária entre o mecanismo DuckDB e o *front-end* da aplicação.

Entre os endpoints previstos estão:

- /api/defeitos;
- /api/pareto;
- /api/scrap;
- /api/scrap/lotes;
- /api/causas;
- /api/causas/turnos.

Os resultados serão retornados em formato JSON, permitindo que o JavaScript realize as requisições e atualize os gráficos dinamicamente.

13 Interface Web

A interface será desenvolvida utilizando HTML5, CSS3 e JavaScript.

O HTML será responsável pela estrutura da aplicação, o CSS pela apresentação visual e responsividade e o JavaScript pela lógica de interação e comunicação com a API.

A biblioteca Chart.js será utilizada para construir as visualizações interativas.

A interface deverá apresentar informações de forma organizada, facilitando a interpretação dos dados por profissionais responsáveis pelo acompanhamento da qualidade.

14 Visualização dos Dados com Chart.js

A camada de visualização da aplicação será desenvolvida utilizando HTML5, CSS3 e JavaScript, com a utilização da biblioteca Chart.js para a construção dos gráficos. Essa

combinação permite separar a estrutura, a apresentação e a lógica da interface, ao mesmo tempo em que possibilita a criação de visualizações interativas a partir dos dados fornecidos pela API.

O HTML5 será utilizado para definir a estrutura dos elementos presentes na página, como títulos, menus, cartões de indicadores, áreas destinadas aos gráficos e demais componentes da interface. Dessa forma, o HTML será responsável pela organização estrutural do dashboard.

O CSS3 será utilizado para definir a apresentação visual da aplicação. Por meio dele serão configurados aspectos como dimensões, espaçamentos, posicionamento dos elementos, tipografia, bordas e responsividade. A utilização do CSS permite organizar os componentes do dashboard de maneira que as informações possam ser apresentadas de forma clara e organizada.

O JavaScript será responsável pela lógica de funcionamento da interface e pela comunicação com a API desenvolvida em FastAPI. Quando o usuário acessar o dashboard ou alterar algum filtro, o JavaScript poderá realizar uma requisição para os endpoints da API, receber os dados em formato JSON e encaminhá-los para os componentes responsáveis pela visualização.

14.1 Justificativa para Utilização do Chart.js

O Chart.js será utilizado como biblioteca de visualização de dados da aplicação devido à sua integração com aplicações desenvolvidas com HTML, CSS e JavaScript. A biblioteca permite transformar os dados obtidos pela API em gráficos que podem ser incorporados diretamente à página web.

A utilização do Chart.js também permite que os gráficos sejam atualizados dinamicamente a partir dos dados retornados pelas consultas realizadas no DuckDB. Dessa forma, a aplicação não precisa gerar imagens estáticas para representar os resultados. Os dados são enviados pela API em formato JSON e utilizados pelo JavaScript para construir ou atualizar os gráficos no navegador.

Essa característica é especialmente importante para o projeto, pois o dashboard deverá permitir diferentes análises dos dados de qualidade. Ao selecionar filtros como modelo do motor, lote, turno ou período, o JavaScript poderá solicitar novos dados à API e atualizar as visualizações apresentadas ao usuário.

Outro fator considerado é a variedade de gráficos necessária para atender aos objetivos do projeto. O Chart.js disponibiliza diferentes tipos de visualização que podem ser utilizados de acordo com a natureza de cada análise.

No projeto serão utilizados principalmente:

- **Gráfico de barras:** utilizado para apresentar a quantidade de ocorrências de cada tipo de defeito;
- **Gráfico de linha:** utilizado para apresentar a evolução da quantidade de peças destinadas ao refugo ao longo dos lotes;
- **Gráfico de barras agrupadas:** utilizado para comparar as causas dos problemas entre os diferentes turnos;
- **Gráfico de Pareto:** construído a partir de um gráfico de barras associado a uma linha de percentual acumulado, permitindo identificar a concentração das ocorrências de defeitos.

A escolha da biblioteca também contribui para manter a arquitetura do projeto relativamente simples. Como o Chart.js funciona na camada de apresentação, não é necessário transferir a responsabilidade de geração dos gráficos para o servidor. A API fica responsável por consultar e fornecer os dados, enquanto o JavaScript utiliza essas informações para construir as visualizações no navegador.

14.2 Integração entre API e Chart.js

O fluxo de comunicação entre as diferentes camadas da aplicação ocorrerá da seguinte maneira:

Parquet → DuckDB → SQL → FastAPI → JSON → JavaScript → Chart.js → Gráfico

Inicialmente, o DuckDB realizará a consulta SQL sobre os dados armazenados no arquivo Parquet. O resultado da consulta será processado pela API desenvolvida com FastAPI e disponibilizado ao *front-end* em formato JSON.

O JavaScript realizará uma requisição HTTP para o endpoint correspondente da API. Após receber a resposta, os dados serão interpretados e utilizados como entrada para o Chart.js.

Um exemplo simplificado dessa comunicação pode ser representado pelo seguinte código:

```
fetch('/api/defeitos')
  .then(response => response.json())
  .then(dados => {
```

```
new Chart(document.getElementById('grafico'), {
  type: 'bar',

  data: {
    labels: dados.map(item => item.tipo defeito),

    datasets: [{
      label: 'Quantidade de defeitos',
      data: dados.map(
        item => item.quantidade
      )
    }]
  }
});

});
```

Nesse exemplo, o JavaScript solicita os dados ao endpoint `/api/defeitos`. A API retorna os registros em formato JSON e o código utiliza os campos retornados para definir os rótulos e os valores do gráfico.

Dessa maneira, existe uma separação clara entre as diferentes responsabilidades do sistema: o DuckDB realiza a consulta dos dados, a FastAPI disponibiliza os resultados, o JavaScript gerencia a interação da interface e o Chart.js realiza a representação gráfica dos dados.

14.3 Aplicação dos Gráficos no Dashboard

A utilização do Chart.js estará diretamente relacionada aos três principais requisitos de visualização definidos para a aplicação.

O primeiro será o **diagrama de Pareto dos defeitos**. Nesse caso, os tipos de defeito serão organizados de acordo com a quantidade de ocorrências, permitindo visualizar quais categorias representam a maior parcela dos problemas identificados.

O segundo será a **evolução do scrap por lote**. Para essa análise, um gráfico de linha poderá representar os lotes no eixo horizontal e a quantidade de peças destinadas ao refugo no eixo vertical.

O terceiro será a **classificação das causas por turno**. Nesse caso, um gráfico de barras agrupadas poderá apresentar as diferentes causas no eixo horizontal e permitir a comparação de suas ocorrências entre o primeiro, segundo e terceiro turnos.

Além dos gráficos, o dashboard poderá utilizar os mesmos dados para apresentar indicadores numéricos, como quantidade total produzida, quantidade de peças inspecionadas, quantidade de defeitos identificados e quantidade total de peças destinadas ao refugo.

14.4 Responsabilidade de Cada Tecnologia

A organização das tecnologias utilizadas na camada de apresentação pode ser resumida da seguinte forma:

Tabela 3: Responsabilidade das tecnologias da interface

Tecnologia	Responsabilidade
HTML5	Estruturação dos elementos da página e organização do dashboard.
CSS3	Estilização, posicionamento, dimensões, responsividade e apresentação visual.
JavaScript	Lógica da interface, interação com o usuário e comunicação com a API.
FastAPI	Disponibilização dos dados processados por meio de endpoints REST.
JSON	Formato utilizado para transportar os resultados das consultas entre a API e o <i>front-end</i> .
Chart.js	Construção e atualização dos gráficos a partir dos dados recebidos pela aplicação.

Fonte: Elaborado pelos autores (2026).

A utilização conjunta dessas tecnologias estabelece uma separação entre a camada de dados e a camada de apresentação. O processamento e as consultas permanecem no lado responsável pelos dados, enquanto o navegador fica responsável pela interação e pela representação visual das informações.

Essa divisão permite que os gráficos sejam atualizados sem a necessidade de modificar diretamente os arquivos Parquet. Quando uma nova consulta é realizada, os dados retornados pela API podem ser utilizados pelo JavaScript para atualizar os componentes visuais do dashboard.

15 Dashboard de Controle de Qualidade

O produto final do projeto será um dashboard web destinado à análise das informações de qualidade da linha de montagem.

O dashboard será composto principalmente por três grupos de visualizações.

15.1 Diagrama de Pareto

O primeiro componente será um diagrama de Pareto destinado a identificar os tipos de defeitos que apresentam maior quantidade de ocorrências.

O gráfico apresentará as categorias de defeito ordenadas de forma decrescente, juntamente com a curva de percentual acumulado.

Essa visualização permitirá identificar quais categorias concentram a maior parte das ocorrências registradas.

15.2 Evolução do Scrap por Lote

O segundo componente será responsável por apresentar a evolução da quantidade de peças destinadas ao refugo ao longo dos lotes.

Essa visualização permitirá observar variações no volume de scrap entre diferentes períodos ou ordens de produção.

O usuário poderá utilizar filtros para comparar modelos de motores, turnos ou outros parâmetros disponíveis no dataset.

15.3 Causas por Turno

O terceiro componente apresentará a distribuição das causas dos defeitos de acordo com o turno de produção.

Essa análise permitirá comparar a ocorrência de diferentes categorias de causa entre os turnos e identificar padrões que possam ser investigados posteriormente.

16 Fluxo de Utilização

O funcionamento da aplicação poderá ser resumido nas seguintes etapas:

1. Os dados simulados são disponibilizados em formato CSV;
2. O Pandas realiza o carregamento e tratamento dos dados;
3. Os dados tratados são armazenados em formato Parquet;
4. O DuckDB executa consultas SQL sobre o arquivo Parquet;
5. A FastAPI disponibiliza os resultados das consultas por meio de endpoints REST;

6. O JavaScript realiza requisições à API;
7. Os dados recebidos em JSON são utilizados para alimentar os gráficos;
8. O usuário interage com o dashboard e realiza filtros para análise dos dados de qualidade.

17 Conclusão

A aplicação proposta apresenta uma arquitetura voltada à transformação de dados de controle de qualidade em informações visuais para apoio à análise de um processo industrial. O cenário adotado considera uma linha de montagem de motores elétricos das famílias W12 e W22, utilizando dados simulados para representar ocorrências de defeitos, retrabalho e refugo.

A utilização do Pandas permite realizar a etapa de preparação dos dados, incluindo limpeza, validação e transformação. Após esse processamento, o formato Parquet proporciona uma estrutura adequada para o armazenamento dos dados tratados e para sua utilização em operações analíticas.

O DuckDB será utilizado para realizar consultas SQL diretamente sobre os dados armazenados em Parquet, permitindo a obtenção das informações necessárias para as análises. A utilização de uma API desenvolvida com FastAPI estabelece uma separação entre a camada de dados e a interface de apresentação, disponibilizando os resultados das consultas em formato JSON.

Na camada de apresentação, HTML5, CSS3 e JavaScript serão utilizados para a construção da interface, enquanto o Chart.js será empregado na elaboração dos gráficos interativos. Como resultado, espera-se obter um dashboard capaz de apresentar o diagrama de Pareto dos defeitos, a evolução do scrap por lote e a distribuição das causas por turno.

Dessa forma, a solução integra diferentes tecnologias em um pipeline único, contemplando desde a entrada e tratamento dos dados até sua consulta e visualização. A arquitetura proposta também permite demonstrar, em uma aplicação prática, conceitos relacionados à engenharia de dados, consultas SQL, desenvolvimento de APIs e visualização de dados aplicados a um contexto de controle de qualidade industrial.

Por se tratar de um cenário acadêmico, os dados utilizados serão simulados. Entretanto, a estrutura desenvolvida poderá servir como modelo conceitual para aplicações semelhantes em ambientes industriais nos quais dados de produção e qualidade precisem ser transformados em informações acessíveis para análise e tomada de decisão.

Referências

- PROPOSTA DE TRABALHO DE PESQUISA. *Visualização de Dados na Indústria 4.0: Engenharia e Design de Interfaces Analíticas na Indústria 4.0*. Material disponibilizado pelo docente. 2026.
- WEG. *Documentação técnica das famílias de motores elétricos W12 e W22*. Documentação técnica utilizada apenas como referência para contextualização do cenário industrial.